

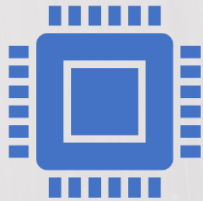
CSDA 5110

Analytics Programming with R

Pradeep Singh

Email –
pradeeps@webster.edu

Objectives



Applied

Write code that creates and modifies variables in R.

Use the Environment pane to view variables in memory.

Code arithmetic expressions.

Interpret R error messages.



Knowledge

Describe the overall purpose of data analysis.

List the five phases of data analysis.

Describe the main uses for RStudio's Source, Console, and Environment panes.

List and describe the four basic data types in R.

What data analytics includes

- Data analysis
- Data visualization (data viz)
- Data modeling (predictive analysis)
- Related terms**
 - Business analytics
 - Sports analytics
 - Data science
 - Data mining
 - Machine learning
 - Deep learning
 - Artificial intelligence (AI)



WHY LEARN R?



R is FREE!



R is Flexible!



R has a HUGE number of resources!

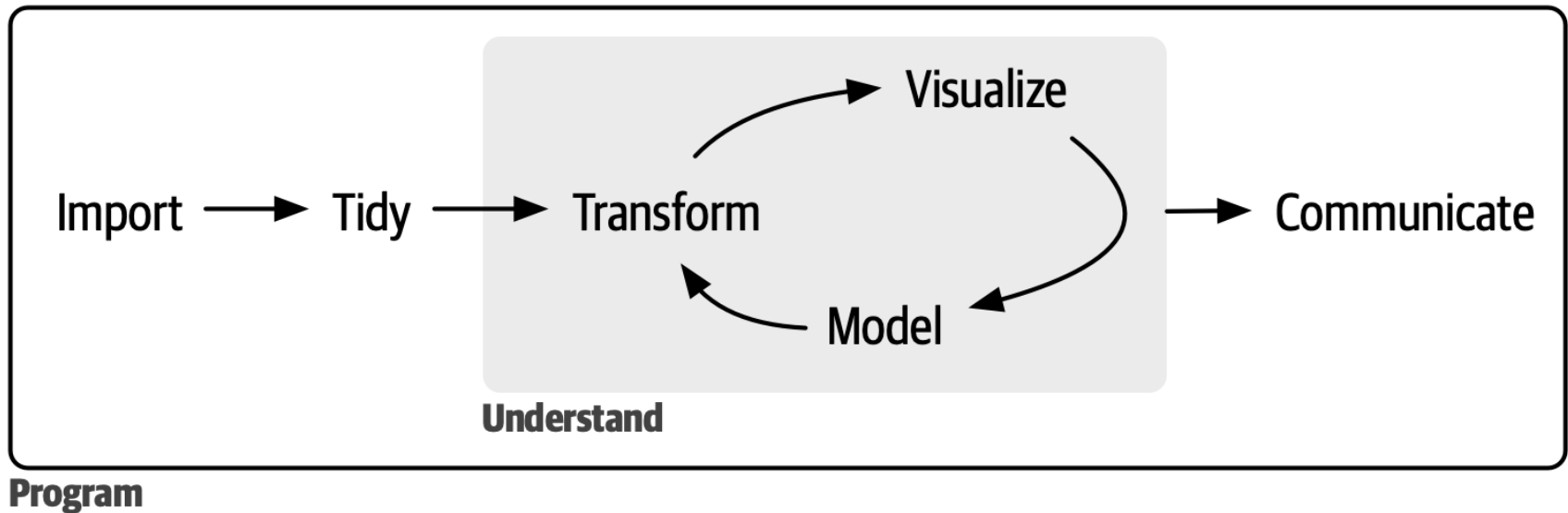
CRAN (Comprehensive R Archive Network) has over 7500 various packages developed by over 4500 developers.

Lots of online blogs and websites devoted to R. Check out R-Bloggers!

WHY LEARN R?

Cutting Edge Technology

- Packages using cutting edge techniques in statistical testing and other data analysis techniques are constantly being developed.



To download R, go to CRAN,
the **c**omprehensive **R** archive **n**etwork, <https://cloud.r-project.org>

RStudio is an integrated development environment, or IDE, for R programming,
which you can download from <https://posit.co/download/rstudio-desktop/>.

Installing R

R supports 32-bit and 64-bit

R can run on Linux, Windows and Mac OS X

Installing R

1

Go to <https://cran.r-project.org/>

2

Select the correct download link for your OS

3

Follow the onscreen prompts and select to use the default settings.

4

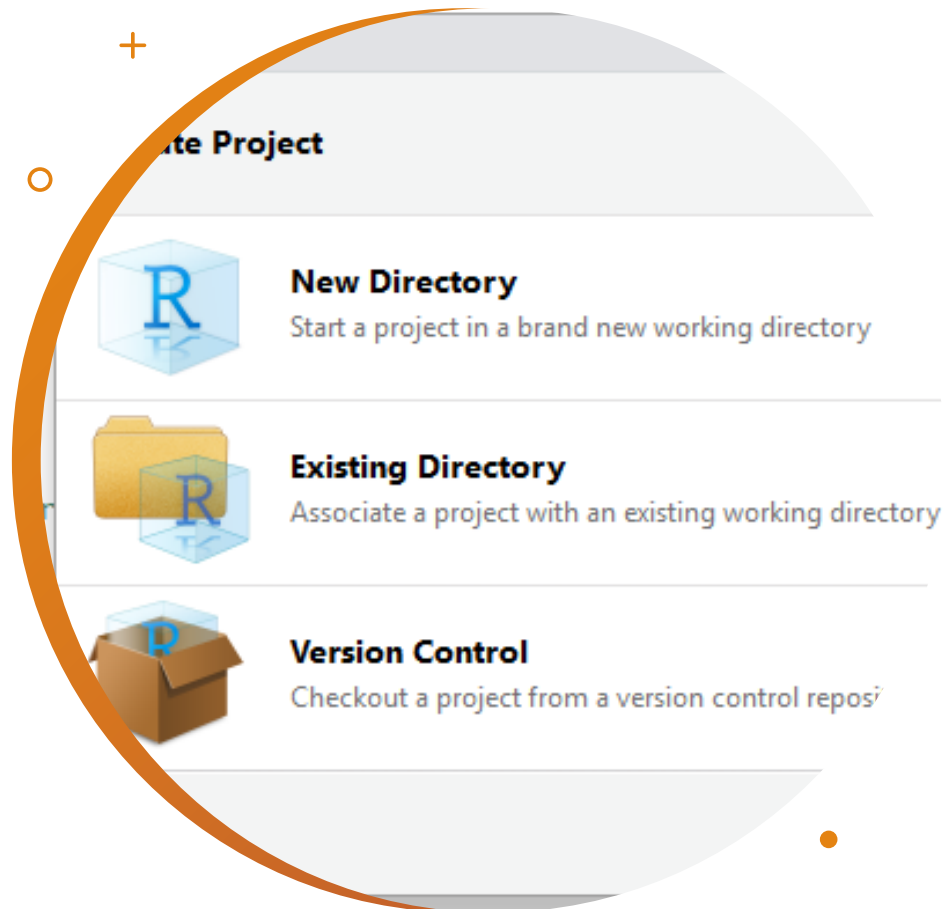
Once R is installed a shortcut should be installed on your desktop. Double click on the R icon to open the command prompt console.

R Studio

RStudio is an IDE (Integrated Development Environment)

Interfaces with R to allow for better script development

- Code formatting
- Syntax Highlighting
- Autocompletion/Autosuggestion of Syntax
- Visualization views
- Package Management
- Debugging and much, much more...
- Download R Studio Free from Posit
[Download RStudio | The Popular Open-Source IDE from Posit](#)



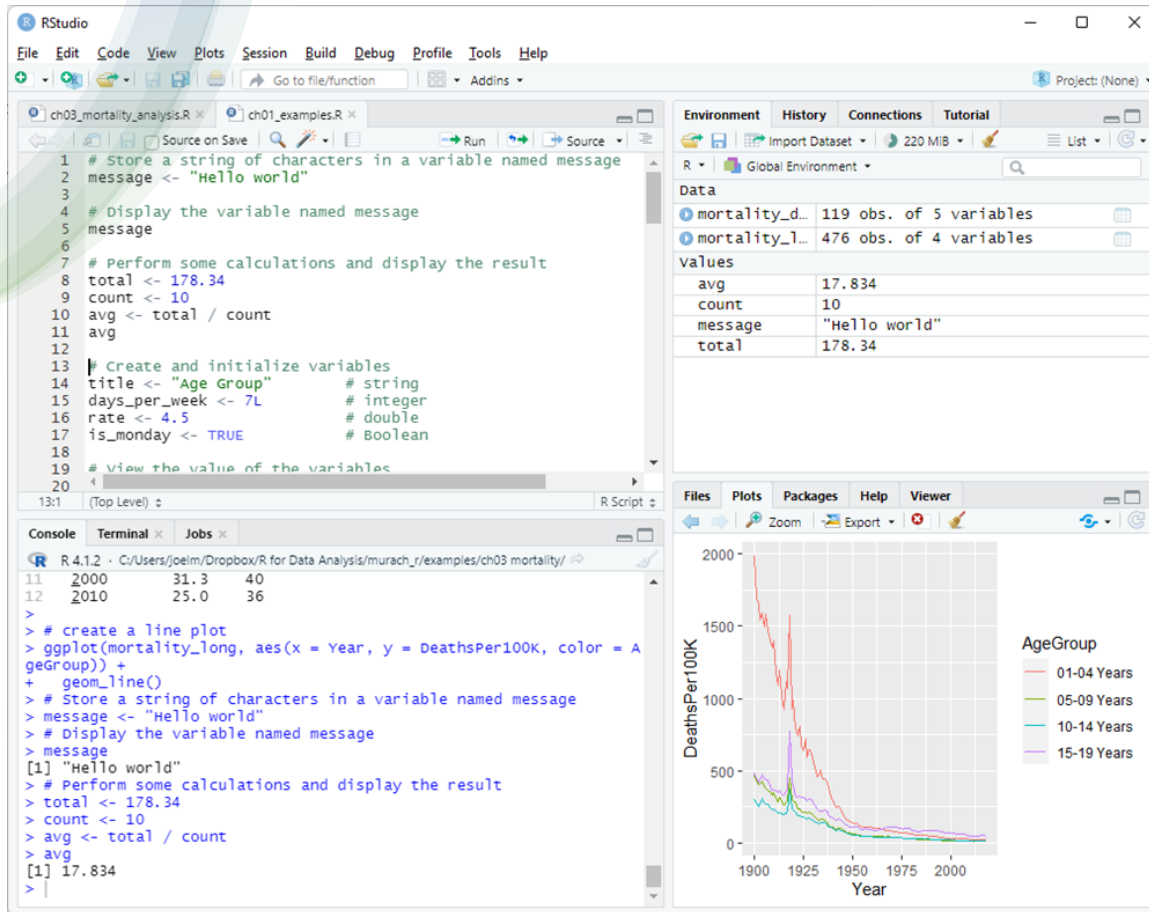
R Studio

Creating a R Project

Creating Projects in R help to organize related files, data and graphs

Every project has its own working directory in R Studio

There are 3 ways to create a new project in R Studio. We will use **New Directory** for each new project.



The RStudio IDE

Two R statements

```
message <- "Welcome to Spring class of CSDA 5110"  
Message
```

- Two comments added to the statements
 - # store a string of characters in a variable named message
 - message <- "Welcome to Spring class of CSDA 5110"
 - # display the variable named message
 - message
-

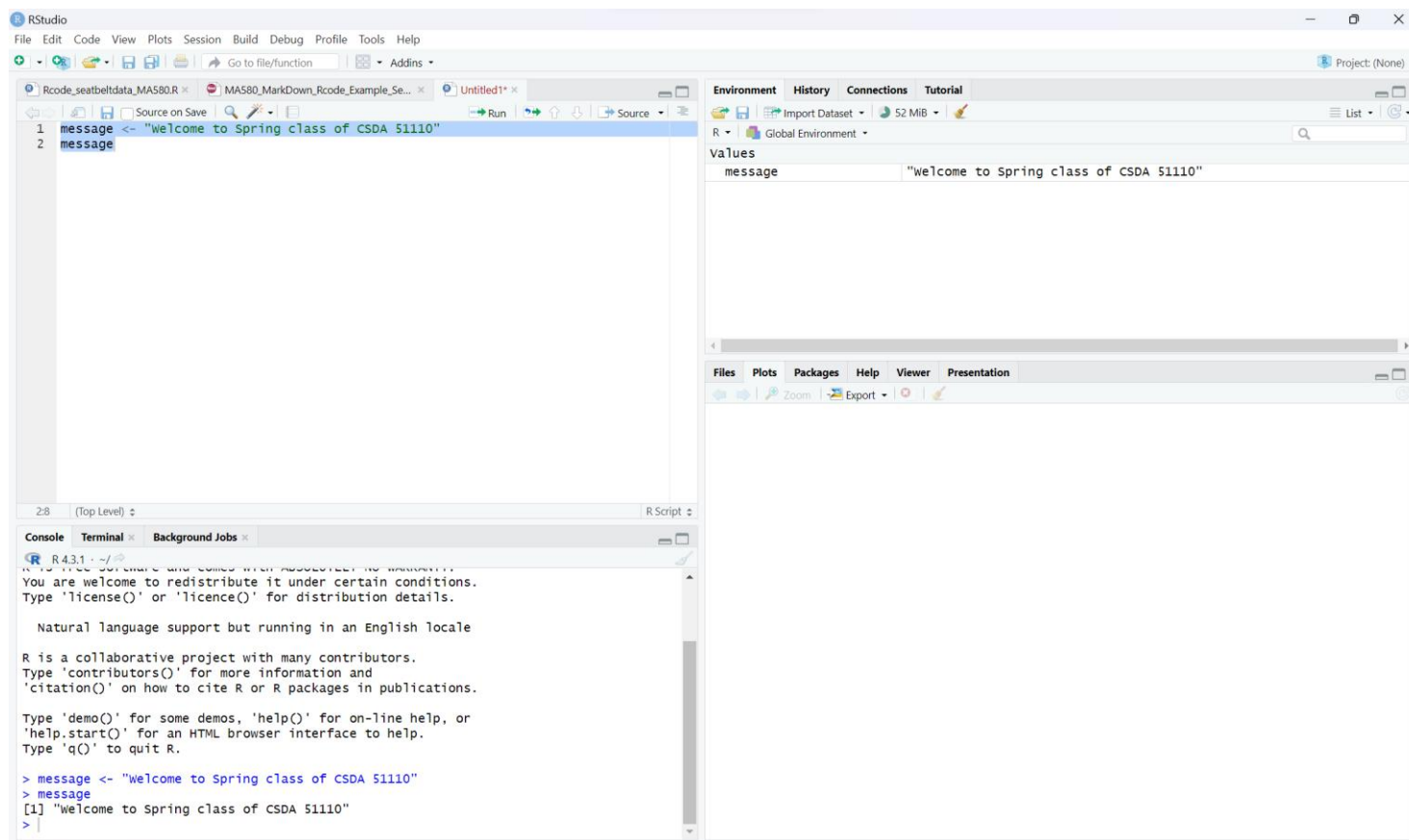
The Console pane

How to use the Console pane

To run a statement in the Console pane, type it after the `>` prompt and press Enter.

If you run a statement that displays output, the Console pane begins each line of output with a number where [1] starts the first line, [2] starts the second line, and so on.

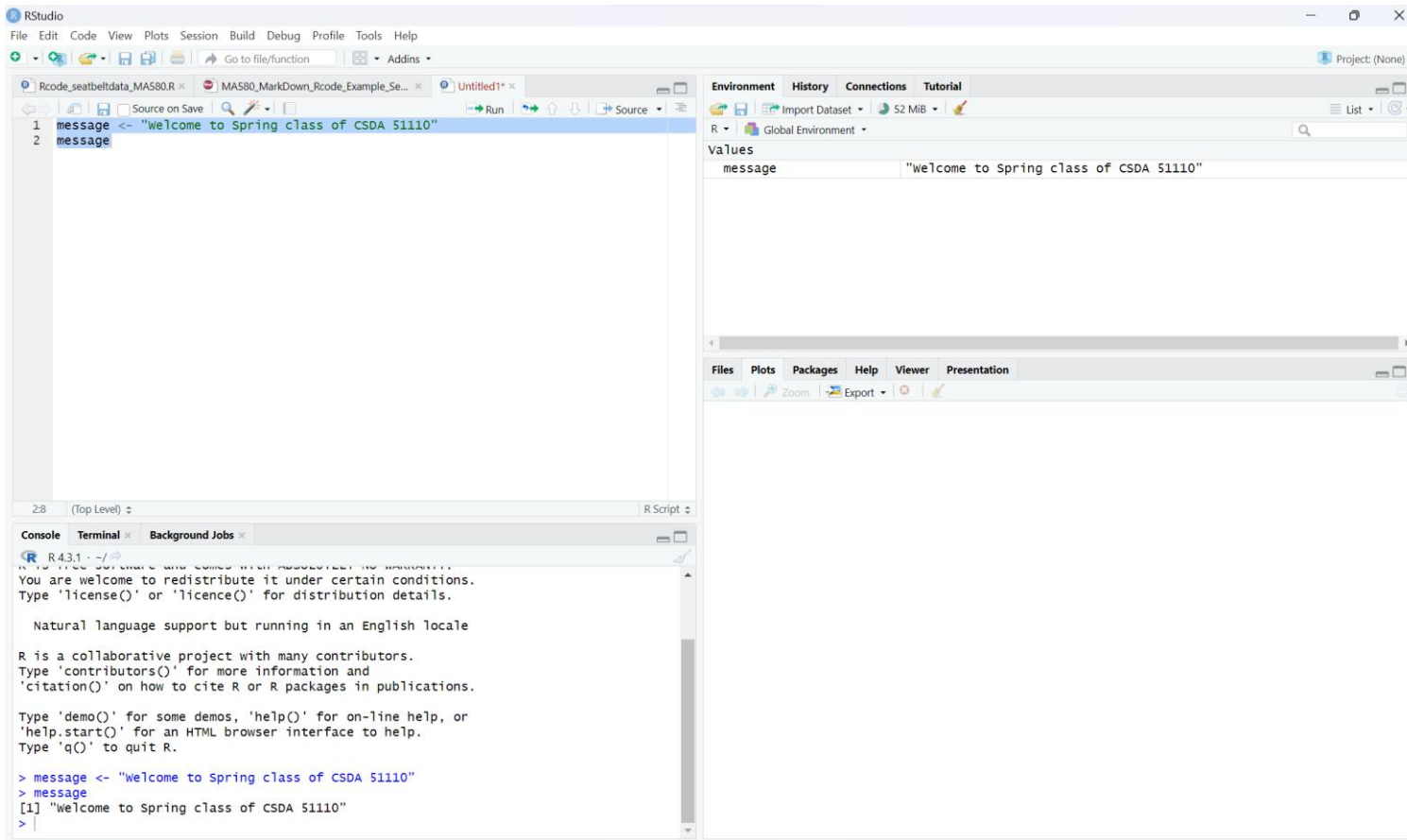
To display previous statements in the Console pane, press the up and down arrows while in the pane. Then, you can re-run the statement that's currently displayed by pressing Enter. Or, you can edit the statement that's displayed before pressing Enter to run it.



The Source and Console panes

How to use the Source pane

- To open an existing R script, select File ->Open File from the menus and use the resulting dialog to select the .R file you want to open.
- To create a new R script, select File ->New File -> R Script from the menus.
- The Source pane displays the tabs for all open files along the top of the pane.
- To switch between open files, click the tab for the file you want.
- To run a statement in a script, move the cursor to the statement and press Ctrl+Enter (Windows) or Command+Enter (macOS).
- To run multiple lines of code, select the lines and press Ctrl+Enter (Windows) or Command+Enter (macOS), or click the Run button in the upper right of the pane.
- The Console pane displays the output of running any statements in the Source pane.



The Environment pane

R Package

An R **package** is a collection of functions, data, and documentation that extends the capabilities of base R.

- In R, the fundamental unit of shareable code is the package.
- A package bundles together code, data, documentation, and tests, and is easy to share with others.
- A "Package" is a library of written code designed to accomplish some task or a collection of tasks.
- Not all packages are of the same quality. Some are more robust than others. Be careful to review the packages that you install.

New packages are being developed continuously (open source)

R Packages

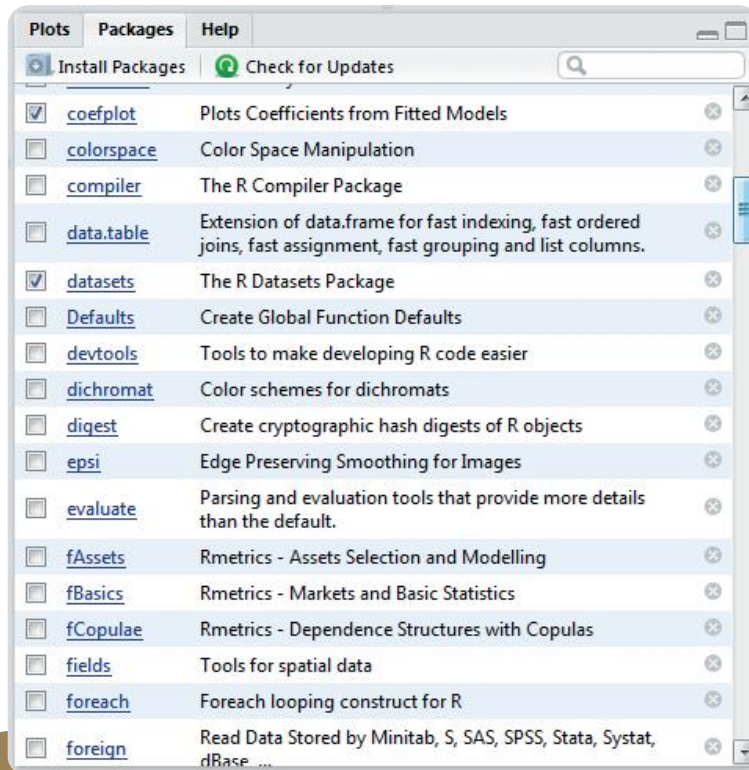
Packages cover a variety of tasks and functions

- Plotting and Data Visualization - GGPLOT2
- Building Web Applications – Shiny
- Data Importing – readr and readxl
- Date and Time manipulation – lubridate

For a list of popular packages go to [Posit | Popular Packages for Open-Source Data Science](#)

Installing Packages

Using install option from package menu in
the bottom right pane



R Packages

The simplest way to install a package is to install them via the GUI in R Studio

Select the packages you want to install by check marking the package and then Install Package in the upper left-hand corner.

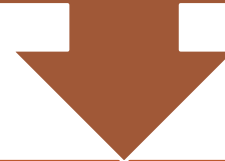
Installing Packages

In the console

```
install.packages("tidyverse")
```



To use the functions within a package the package must be loaded. This is done using the function "library()" or the function "required()" to define the package to be used in your code.



Using the function "library()" is the preferred method

Loading Package

You will not be able to use the functions, objects, or help files in a package until you load it

[library\(tidyverse\)](#)

```
> library(tidyverse)
```

```
— Attaching core tidyverse packages
```

```
tidyverse 2.0.0 —
```

```
✓ dplyr  1.1.0  ✓ readr  2.1.4
```

```
✓ forcats 1.0.0  ✓ stringr 1.5.0
```

```
✓ ggplot2 3.4.1  ✓ tibble  3.1.8
```

```
✓ lubridate 1.9.2  ✓ tidyr   1.3.0
```

```
✓ purrr   1.0.1
```

```
— Conflicts
```

```
— tidyverse_conflicts() —
```

```
✗ dplyr::filter() masks stats::filter()
```

```
✗ dplyr::lag()   masks stats::lag()
```

```
ℹ Use the conflicted package to force all conflicts to become errors
```

```
Warning message:
```

```
package ‘tidyverse’ was built under R version 4.2.3
```

R Packages



Calling a package that has not been loaded will produce an error.

To unload a package, use the `"detach(package:package_name)"` function

- Note – many packages do the same thing and may use the same function names. Using `detach(package: ...)` removes all instances of the package so that R doesn't get confused and use the wrong function.



R Basics



In order to be proficient with R you have to know the basics of R.

- Basic Math – Order of Precedence
 - Variables – How to create and use them
 - Data Types
 - Vectors – The basic building block of R
 - Functions – How to create and call them in R
 - Missing Data – How to handle
 - Pipes
-

R Basics

R Basics – Variable

A Variable is a type of container. It can hold numbers, text, Boolean values, functions and results.

Unlike other programming languages, variables do not have to be declared and are not required to hold only one data type throughout.

To create a variable in R the variable is "assigned" a value using `<-` or `=`. The preferred method is `<-`

- Ex. `x <- 2` or `2 -> x`

Variable assignments can be done simultaneously.

- Ex. `x <- 2 y <- 8`

R Basics

R Basics

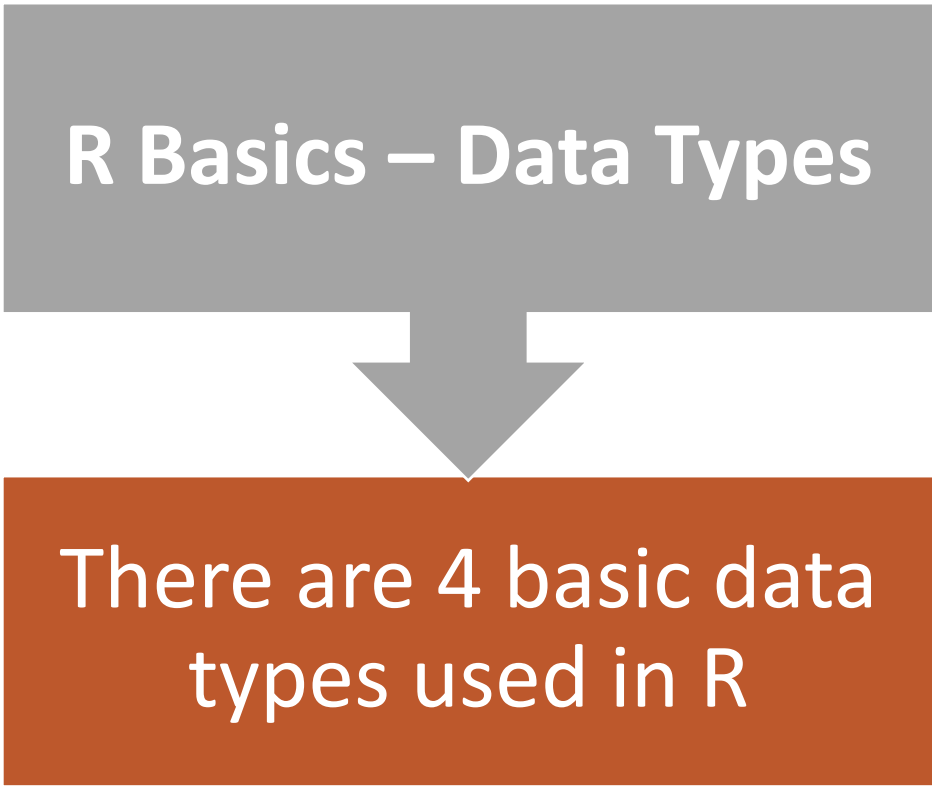
R Basics – Variable

Variable names should be something meaningful and typically use camel case where the first letter of the first word is lowercase.

Ex of camel case "data_seatbelt"

R Basics

R Basics – Data Types



```
graph TD; A[R Basics – Data Types] --> B[There are 4 basic data types used in R]
```

There are 4 basic data types used in R

The four basic data types in R

Type	Name	Literal values
<code>character</code>	String	<code>"x", "abc", "123", "this is a character string"</code>
<code>integer</code>	Integer	<code>1L, -3L, 5L</code>
<code>double</code>	Double	<code>12.3, 5.0, -3.562, 7</code>
<code>logical</code>	Boolean	<code>TRUE, FALSE</code>

R Basics

Numeric Data



Numeric Data

The most commonly used data type in R

Numeric values stored in R are automatically assumed to be of the Numeric data type.

Testing if a variable is numeric is done with the "is.numeric" function. It will return a TRUE or FALSE value

R Basics

Numeric Data

Numeric Data

Integers are a type of numeric and are denoted using the letter "L" after the value assigned.

- Ex. `a <- 7L` (use the "is.integer" function to check. It will also pass the "is.numeric" check as well.
- Integers are "promoted" to numeric when needed automatically

R Basics

Character Data

Character Data

R handles character data in one of two ways

- Character
- Factors – This will become important as we look at Data Frames.

To find the length of a character string in a variable use the "nchar()" function

R Basics

Logical – Boolean Data

Logical – Boolean Data

Logicals can result in comparing two numbers or characters

Use the logical operators to evaluate numbers or characters

- "==" (Equal to) or "!=" (Not equal to)
- < or > (Greater than or Less than)
- <= or >= (Greater than or equal to or Less than or equal to)

Will always return a TRUE or FALSE

Use arithmetic
on logical
values

```
true_count <- FALSE + TRUE +  
FALSE + TRUE + FALSE
```

```
true_count
```

```
[1] 2
```

R Basics

Vectors

Vectors

Vectors are the basic building blocks of R

A vector is a collection of elements of the same data type

- Vectors can only be of one data type also known as "atomic".

Vectors are powerful in that operations are applied to each element in the vector automatically versus having to loop through each element

R Basics

Vectors

Vectors

There are no dimensions to vectors, meaning there is no concept of a row vector or column vector.

Vectors are created using the "c()" function which stands for "Combine".

The first element's index in a vector is 1 and not 0 like other programming languages

R Basics

Functions

Functions

We have already covered a few of the functions available in R

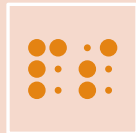
- `nchar()`
- `length()`
- `as.Date()`
- `is.numeric()`
- `is.character()`

R Basics

Functions



Functions



Any function provided in R has accompanying documentation. The easiest way to access that documentation is to place a question mark in front of the function name, like this: `?mean`.



To get help on binary operators like `+`, `*` or `==` surround them with backticks (```). Ex. `?`==``

R Basics

Missing Data

NA

Missing Data – NA

Having NA values in your vector will result in NA for the result if using mathematical functions.

```
◦ z <- c(1, 2, NA, 8, 3, NA, 3)
> z
[1] 1 2 NA 8 3 NA 3
> mean(z)
[1] NA
```

When using a functions like mean(), sum(), min(), max(), var(), and sd() you can use the argument “rm.na = TRUE” to removed NA values.

```
◦ mean(z, na.rm=TRUE)
[1] 3.4
```

R Basics

Missing Data

NA

Missing Data – NA

NULL represents nothingness. It's not missing data, there is simply nothing there to return.

NULL values cannot exist in a vector. They disappear

- Ex.

```
> z <- c(1, NULL, 3)
```

```
> z
```

```
[1] 1 3
```


R Basics

Missing Data

NULL

Missing Data – NA

The test for NULL values is “is.null()” which returns a TRUE value if NULL is present and FALSE if not present.

```
◦ Ex. > x <- NULL
      > is.null(x)
      [1] TRUE
      > y <- 81
      > is.null(y)
      [1] FALSE
```

R Basics

Data Frames



One of the most powerful features in R, are Data Frames.



Data Frames are like Excel Spreadsheets.

Data Frames

Data Frames are like Excel
Spreadsheets

Each Column is a Variable and a Vector.
Each vector is of the same length.

Each column holds the same data type
within the column, but each column
can be different data types.

Each Row is an observation

Variable A		
88.5	Red	True
90.3	Green	False
55.7	Yellow	True
125.0	Blue	True



Data Frames

Data Frames are created using the `"data.frame()"` function

Use `"nrow()"` to check the number of rows in a data frame

Use `"ncol()"` to check the number of columns in a data frame

Use `"dim()"` to check the number of rows and columns in a data frame



Data Frames

Use `"names()"` to look at the column names of a data frame

Use `"rownames()"` to check the names of each row.

Use `rownames() <- c("One", "Two", "Three", ect)` to assign names to the individual rows in a data frame

Data Frames



To look at the first few rows of a data frame use the "head()" function.

To look at the last few rows of a data frame use the function "tail()"

You can use the "class()" function to check the class of a data frame.

Data Frames

The `$` or `[]` is used to access individual columns of a data frame.

- Ex. `data_seatbelt$2016`
- Ex. `Data_seatbelt[3, 2]` # the row is the first argument and the column is the second argument

To access multiple columns by name make the column argument a character vector of names

- `Data_seatbelt[, c("2015", "2016")]`

Data Frames

When using the `[ ]` there is a third argument that can be used to ensure you return a single column, `"drop = FALSE"`

- `Data_seatbelt[ , "2019", drop = FALSE]`